

CSA1511-cloud computing

S.Harshavardhan
CH.Rohith
B.Balaji

CLOUD COMPUTING – ASSIGNMENT REPORT

Particular	Details
Department:	Computer Science / Information Technology
Programme:	B.Tech / B.E. – Computer Science / Information Technology
Course Code & Course Name	Cloud Computing
Academic Year / Batch	2026–2027
Faculty Name	<u>Dr.R.Hemavathi</u>
Assignment Title	Cloud-Based Property Buying and Rental Management System
Date of Issue	<u>2-9-26</u>
Date of Submission	<u>2-9-26</u>
Maximum Marks	100
Course Outcome(s) – CO	CO3 and CO4 – Apply cloud concepts and develop/deploy cloud-based applications
Bloom's Taxonomy Level	L4 – Analyze; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Digital property discovery, rental management, customer enquiry handling and secure cloud-based real-estate services

B. Assignment Problem / Challenge

Design, develop, and deploy a cloud-based application for managing the property buying and rental process in a selected city/region. The application demonstrates the Software as a Service (SaaS) model and provides facilities for property listing, searching, buying, renting, commercial property management, rental agreements, and customer enquiries. Suitable fields include property type, transaction type, location, area, locality, price/rental range, property value, amenities, and agreement details.

For this implementation, Chennai, Tamil Nadu, India is selected as the target region and Amazon Web Services (AWS) is selected as the cloud service provider. The proposed system uses cloud application hosting, managed relational storage, object storage, identity/access controls and monitoring.

- Apply cloud computing concepts to a practical SaaS application.
- Analyze functional and non-functional requirements.
- Design a scalable cloud architecture and relational database.
- Implement listing, searching, buying/renting, agreement and enquiry modules.
- Demonstrate authentication, authorization, validation and secure configuration.
- Test major functions and document results using screenshots.
- Upload source code to GitHub and provide the real repository link.

C. Problem Statement

Problem / Case / Design Challenge:

Traditional property buying and rental processes often require users to search through disconnected advertisements, contact multiple agents, compare properties manually, and manage enquiries and rental information separately. The proposed solution is a centralized cloud-based SaaS application that allows owners/agents to publish listings and customers to search, filter, enquire about, buy or rent properties through a single web interface.

The system supports residential and commercial properties in Chennai. It maintains property details, availability, price/rent, amenities, customer information, transaction status, rental agreement information and enquiries. The application is designed so that additional properties, users and locations can be added without redesigning the core system.

D. Requirements and Constraints

Functional Requirements

- User registration/login and role-based access.
- Add, update, view and remove property listings.
- Property types: Apartment, Villa, Independent House, Plot, Office, Shop, Warehouse, etc.
- Transaction types: Buy, Rent and Lease.
- Search/filter by city, locality, property type, transaction type, area and price/rent range.
- Display amenities such as parking, lift, security, power backup and furnished status.

- Record property value, selling price, monthly rent and deposit.
- Customer enquiry submission for a selected property.
- Purchase/rental transaction status management.
- Rental agreement records containing tenant, owner, dates, deposit, monthly rent and terms.
- Commercial property management.
- Cloud object storage for property images/documents in production.
- Administrator management of listings and enquiries.

Non-Functional Requirements

- Web accessibility through a browser.
- Responsive search and normal-load performance.
- Scalability for growth in users and properties.
- Password hashing and role-based access.
- Database backup and reliable object storage.
- Maintainable separation of application, data and configuration.
- Simple and understandable forms.
- Protection of personal and agreement information.

Constraints

- Cloud cost must remain within the student/project budget.
- Internet connectivity is required.
- Real payment gateway and legally binding e-signature are outside the prototype scope.
- Property ownership/legal verification is outside the prototype.
- Production deployment requires valid cloud credentials, HTTPS, secure networking and monitoring.

E. Student Work

1. Problem Understanding and Formulation

What is the problem?

Create a unified property marketplace and management service that reduces the effort involved in finding properties, publishing listings, handling enquiries and managing rental agreements.

Expected Outcomes

A working web application, searchable property catalogue, listing management, buy/rent workflow, agreement records, enquiry management, cloud-ready architecture and security controls.

Information/Data Available

Property ID, title, description, type, transaction type, city, locality, address, area, price, rent, deposit, value, bedrooms, bathrooms, furnishing, amenities, availability, owner/agent, enquiries and agreements.

Assumptions

Chennai is the selected region; users have internet access; sample data is valid for demonstration; legal verification is outside scope; AWS is the target cloud provider.

Constraints to be Satisfied

Authenticated access for protected functions, authorization for management actions, valid property/enquiry relationships, correct search filters and protected configuration.

2. Application of Course Knowledge

The solution applies SaaS, cloud computing, three-tier architecture, REST-style operations, relational database design, authentication/authorization, object storage, scalability, availability, backup and monitoring concepts.

Concept	Application
SaaS	Users access the common property service through a web browser without managing infrastructure.
Cloud Computing	Application hosting, database and storage are provided by AWS services.
Three-tier architecture	Presentation, application/API and data layers are separated.
Database modelling	Users, properties, transactions, agreements and enquiries use linked relational tables.
Security	Authentication, role-based authorization, HTTPS and protected configuration are applied.
Object storage	Property images and documents are stored separately from relational records.
Scalability	Stateless application instances can be scaled horizontally when needed.

Database Design

Table	Primary Key	Selected Fields	Relationship
users	user_id	name, email, password_hash, role	Customer/owner/agent/admin
properties	property_id	title, type, transaction_type, city, locality, area_sqft, price, rent	Belongs to owner
property_amenities	amenity_id	property_id, amenity_name	Many amenities per property
transactions	transaction_id	property_id, customer_id, type, amount, status	Customer ↔ property
rental_agreements	agreement_id	property_id, tenant_id, start_date, end_date, rent, deposit	Property ↔ tenant
enquiries	enquiry_id	property_id, customer_id, message, status	Customer ↔ property
property_media	media_id	property_id, object_key	References S3 object

Major User Flow

1. Open the SaaS application.
2. Register/login.
3. Select Buy, Rent or Lease.
4. Choose location/locality and filters.
5. Application queries the property database.
6. Matching properties are displayed.
7. Review property details, amenities and price/rent.
8. Submit an enquiry or transaction request.
9. Owner/agent/admin reviews and updates status.
10. For rentals, create the agreement record.
11. Update property availability after transaction.

4. Use of Modern Tools

Tool / Technology	Use in Project	Evidence
Python	Backend implementation	Source code and execution
Flask	Web framework	Running application
HTML/CSS	User interface	Browser screenshots
MySQL / SQLite	Relational storage	Database/results
AWS EC2 / Elastic Beanstalk	Cloud hosting	Deployment screenshot

Amazon RDS	Managed database	RDS console screenshot
Amazon S3	Image/document storage	S3 console screenshot
GitHub	Version control/source sharing	Repository screenshot/link

5. Results and Validation

The prototype is validated through functional test cases. The screenshot placeholders below must be replaced by screenshots from the team's actual execution/deployment. This report does not fabricate deployment or GitHub results.

Test ID	Function	Input / Condition	Expected Result	Actual Result
TC01	User Login	Valid credentials	Dashboard opens	To be filled after execution
TC02	Add Property	Valid property form	Property saved	To be filled after execution
TC03	Property Search	Rent + Chennai + locality	Matching properties shown	To be filled after execution
TC04	Price Filter	Minimum/maximum price	Only matching records shown	To be filled after execution
TC05	Property Details	Select listing	Full details shown	To be filled after execution
TC06	Enquiry	Valid message	Enquiry stored/confirmed	To be filled after execution
TC07	Rental Agreement	Valid agreement data	Agreement created	To be filled after execution
TC08	Authorization	Customer tries admin action	Access denied	To be filled after execution
TC09	Invalid Input	Missing required field	Validation message	To be filled after execution

Execution Screenshots / Output

[INSERT ACTUAL SCREENSHOT 1: HOME/LOGIN PAGE]

Figure 2. Home/Login page

[INSERT ACTUAL SCREENSHOT 2: PROPERTY LISTING PAGE]

Figure 3. Property listing page

[INSERT ACTUAL SCREENSHOT 3: SEARCH/FILTER RESULTS]

Figure 4. Search/filter results

[INSERT ACTUAL SCREENSHOT 4: PROPERTY DETAILS WITH AMENITIES AND PRICE/RENT]

Figure 5. Property details with amenities and price/rent

[INSERT ACTUAL SCREENSHOT 5: ADD/EDIT PROPERTY FORM]

Figure 6. Add/Edit property form

[INSERT ACTUAL SCREENSHOT 6: CUSTOMER ENQUIRY PAGE]

Figure 7. Customer enquiry page

[INSERT ACTUAL SCREENSHOT 7: RENTAL AGREEMENT MANAGEMENT]

Figure 8. Rental agreement management

[INSERT ACTUAL SCREENSHOT 8: COMMERCIAL PROPERTY MANAGEMENT]

Figure 9. Commercial property management

[INSERT ACTUAL SCREENSHOT 9: AWS DEPLOYMENT CONSOLE]

Figure 10. AWS deployment console

[INSERT ACTUAL SCREENSHOT 10: DATABASE/STORAGE CONFIGURATION]

Figure 11. Database/storage configuration

6. Analysis and Engineering Decision

A cloud SaaS architecture was selected because multiple customers should access a common service through the web. A managed database reduces operational work compared with maintaining a database manually on the application server. Object storage is separated from relational records because images and documents have different storage and access characteristics.

Decision	Option Considered	Selected	Reason
Application hosting	Local server / cloud VM	AWS cloud hosting	Remote access and scalability
Database	Local SQLite / managed DB	RDS MySQL for production	Multi-user persistent storage
Media storage	Database BLOB / S3	S3	Scalable object storage
Access control	Application-only / IAM + roles	Both	Defense in depth

Advantages

- Centralized property catalogue.
- Accessible through the internet.
- Supports multiple roles.
- Scalable cloud storage.

- Can extend to more cities and categories.
- Managed services reduce infrastructure administration.

Limitations

- No legal ownership verification.
- No real payment gateway in prototype.
- No production e-signature integration.
- Production security/monitoring requires further hardening.
- Sample records are not a live market database.

Trade-offs

Managed cloud services improve reliability and scalability but can increase operating cost. SQLite is convenient for initial development, while RDS MySQL is more appropriate for a multi-user cloud deployment. S3 improves media storage design but adds cloud configuration and access-control requirements.

7. Broader Considerations

Area	Consideration	Project Approach
Sustainability	Efficient resource usage	Use managed services and scale according to demand
Environment	Reduce dedicated local infrastructure	Cloud consolidation
Society	Improve access to property information	Central searchable platform
Safety	Protect user/agreement information	Authentication, authorization and HTTPS
Ethics	Reduce misleading information	Admin review/reporting mechanisms
Economics	Control operating cost	Small development resources and monitoring
Accessibility	Simple interface	Clear labels, filters and responsive design

8. Conclusion

The proposed Cloud-Based Property Buying and Rental Management System provides a practical SaaS solution for property discovery and buying/rental workflows in Chennai. The architecture integrates a web application, relational cloud database, object storage, security controls and monitoring. The major modules cover property listing, searching, buying, renting, commercial property management, rental agreements and customer enquiries.

Possible improvements include online payments, digital signatures, map-based search, recommendation systems, document verification, real-time notifications, analytics, auto-scaling and CI/CD.

9. Student Reflection

This assignment provides practical learning by connecting cloud computing concepts with a complete application workflow. The major learning outcomes include SaaS architecture, separation of application and storage layers, database modelling, access control, cloud deployment planning, testing and technical documentation.

With additional time, the project can be improved using automated tests, CI/CD, production authentication, maps, payments, electronic agreements, monitoring, load testing and automated backup/recovery validation.

10. References

1. Amazon Web Services (AWS) documentation – EC2, RDS, S3, IAM and CloudWatch.
2. Python documentation.
3. Flask documentation.
4. MySQL documentation.
5. GitHub documentation.
6. Course notes and laboratory material for Cloud Computing.

11. Implementation / Source Code

The following compact Flask implementation demonstrates the core prototype. It can be executed locally first and adapted for AWS deployment. For production, replace SQLite with Amazon RDS MySQL and use S3 for uploaded media.

```
# app.py
from flask import Flask, request, redirect, url_for, render_template_string, session
import sqlite3

app = Flask(__name__)
app.secret_key = "CHANGE_THIS_IN_PRODUCTION"
DB = "property.db"

def db():
    con = sqlite3.connect(DB)
    con.row_factory = sqlite3.Row
    return con

def init_db():
    con = db()
    con.executescript("""
CREATE TABLE IF NOT EXISTS properties(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

title TEXT NOT NULL, property_type TEXT NOT NULL,
transaction_type TEXT NOT NULL, city TEXT NOT NULL,
locality TEXT NOT NULL, area REAL NOT NULL,
price REAL DEFAULT 0, rent REAL DEFAULT 0,
value REAL DEFAULT 0, amenities TEXT,
available INTEGER DEFAULT 1

```

```
);
```

```

CREATE TABLE IF NOT EXISTS enquiries(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  property_id INTEGER, customer_id INTEGER,
  message TEXT NOT NULL, status TEXT DEFAULT 'New'

```

```
);
```

```

CREATE TABLE IF NOT EXISTS rental_agreements(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  property_id INTEGER, tenant_id INTEGER,
  start_date TEXT, end_date TEXT, monthly_rent REAL,
  deposit REAL, terms TEXT

```

```
);
```

```
""")
```

```
con.commit(); con.close()
```

```
@app.route("/")
```

```
def home():
```

```
    con=db()
```

```
    props=con.execute("SELECT * FROM properties WHERE available=1 ORDER BY id
DESC").fetchall()
```

```
    con.close()
```

```
    return render_template_string("""
```

```
<h1>Chennai Property SaaS</h1>
```

```
<a href="/add">Add Property</a> | <a href="/search">Search</a>
```

```
<h2>Available Properties</h2>
```

```
{% for p in properties %}
```

```
<div style="border:1px solid #ccc;padding:10px;margin:10px">
```

```
<b>{{p['title']}}</b><br>
```

```
{{p['property_type']}} | {{p['transaction_type']}} |
```

```
{{p['locality']}}, {{p['city']}} | {{p['area']}} sq.ft<br>
```

```
Price: ₹{{p['price']}} | Rent: ₹{{p['rent']}}<br>
```

```
Amenities: {{p['amenities']}}
```

```
</div>
```

```
{% endfor %}
```

```
""", properties=props)
```

```
@app.route("/add", methods=["GET","POST"])
```

```

def add():
    if request.method=="POST":
        con=db()
        con.execute("""INSERT INTO properties
        (title,property_type,transaction_type,city,locality,area,price,rent,value,amenities)
        VALUES(?,?,?,?,?,?,?,?,?,?)""", (
            request.form["title"], request.form["property_type"],
            request.form["transaction_type"], request.form["city"],
            request.form["locality"], float(request.form["area"]),
            float(request.form.get("price") or 0),
            float(request.form.get("rent") or 0),
            float(request.form.get("value") or 0),
            request.form.get("amenities", "")))
        con.commit(); con.close()
        return redirect(url_for("home"))
    return """<h1>Add Property</h1>
    <form method="post">
    Title <input name="title" required><br>
    Type <input name="property_type" required><br>
    Transaction <select name="transaction_type">
    <option>Buy</option><option>Rent</option><option>Lease</option></select><br>
    City <input name="city" value="Chennai" required><br>
    Locality <input name="locality" required><br>
    Area (sq.ft) <input name="area" type="number" required><br>
    Price <input name="price" type="number"><br>
    Rent <input name="rent" type="number"><br>
    Property Value <input name="value" type="number"><br>
    Amenities <input name="amenities"><br>
    <button>Add Property</button></form>"""

```

```

@app.route("/search")

```

```

def search():
    city=request.args.get("city","").strip()
    locality=request.args.get("locality","").strip()
    ptype=request.args.get("property_type","").strip()
    ttype=request.args.get("transaction_type","").strip()
    sql="SELECT * FROM properties WHERE available=1"; args=[]
    if city: sql += " AND city LIKE ?"; args.append("%"+city+"%")
    if locality: sql += " AND locality LIKE ?"; args.append("%"+locality+"%")
    if ptype: sql += " AND property_type LIKE ?"; args.append("%"+ptype+"%")
    if ttype: sql += " AND transaction_type=?"; args.append(ttype)
    con=db(); rows=con.execute(sql,args).fetchall(); con.close()
    return render_template_string("<h1>Search Results</h1>{% for p in rows

```

```
%}<p><b>{{p['title']}}</b> - {{p['locality']}} - {{p['transaction_type']}} - ₹{{p['price']}} /
₹{{p['rent']}}</p>{% endfor %}", rows=rows)
```

```
@app.route("/enquiry/<int:property_id>", methods=["POST"])
def enquiry(property_id):
    message=request.form["message"]
    customer_id=session.get("user_id",1)
    con=db()
    con.execute("INSERT INTO enquiries(property_id,customer_id,message) VALUES(?,?,?)",
                (property_id,customer_id,message))
    con.commit(); con.close()
    return "Enquiry submitted successfully."

if __name__=="__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000, debug=True)
```

12. Pseudocode

BEGIN

Initialize application and connect to database

Create required tables if they do not exist

WHILE application is running

IF home page requested

Retrieve available properties

Display property cards

ENDIF

IF Add Property submitted

Validate required fields

Store property record

Display success

ENDIF

IF Search submitted

Build query from city/locality/type/transaction filters

Retrieve matching records

Display results

ENDIF

IF Enquiry submitted

Validate property and message

Store enquiry linked to customer and property

Display confirmation
ENDIF

IF rental transaction completed
Create rental agreement
Update property availability
ENDIF

Apply authentication and authorization to protected operations
ENDWHILE
END

Assessment Rubric – Reference Format

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

CO–PO–Assessment Mapping – Suggested

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO3	PO1, PO2	L3/L4	10
Application of knowledge	CO3	PO1, PO2, PO5	L3/L4	20
Solution / Design / Implementation	CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4	PO5	L3/L4	10
Validation	CO4	PO4, PO5	L4/L5	15
Analysis & justification	CO4	PO2, PO3	L4/L5	15
Broader considerations	CO4	PO6, PO7	L4/L5	5

Documentation & reflection	CO3/CO4	PO10	L3/L4	5
-------------------------------	---------	------	-------	---